

Compiling by exact queries over index sets

A validated answer to the Luminal compiler challenge,
and evidence for a different way of computing

Alberto Hernández-Espinosa

September 2026

Abstract

This paper has two aims. The first is practical: to give a working, validated answer to the Luminal compiler challenge, which asks a compiler to schedule a straight-line program on a small very-long-instruction-word machine and to place its values in a shared scratchpad, minimising both the number of cycles and the scratch footprint. The second, which matters more to us, is to offer evidence that a different way of computing is worth pursuing now that accelerators expose every scheduling decision to the compiler and compilation is increasingly searched or learned. In the compiler we describe, every scheduling and allocation choice is the witness of an exact query over sets of decision indices, written as cubes, and every later improvement is found by reopening related groups of decisions, deleting impossible options with a replayable certificate, and searching the remainder exactly. The index-set method is work in progress, grounded in the author’s published research on algorithmic information dynamics, and we report the whole programme, including the stages that did not work. The final proposal, C1, makes the same decisions as its predecessor R0 when both are given the same search work: 1,000 of 1,000 paired runs on 200 fresh programs are identical. At that fixed work, of 10,000 search nodes, C1’s complete compile call takes 0.574 of R0’s time (97.5% interval [0.552, 0.598]), and at a wall-clock allowance of 0.1 s its output is not worse within the registered margin (J ratio 0.991, 97.5% interval [0.985, 0.996]). On the eight public programs, which were visible during development, the method scores 2.187957 against 1.901379 for a classical list scheduler with first-fit allocation. That gain is paid for in compile time; the classical compiler remains far faster to compile, and on a fresh population it still wins on some programs. We also report that, under a real clock, a faster search can end at a worse local optimum, a form of path dependence that any anytime local search inherits. The acceptance of C1 was a self-review.

1 Introduction

Accelerators built for machine learning have returned the compiler to the centre of performance. Their cores are frequently very-long-instruction-word (VLIW) machines, in which several functional units issue in the same cycle and the hardware makes no scheduling decisions of its own; every placement in time and in memory is the compiler’s responsibility [22]. At the same time the programs to be compiled are increasingly generated rather than written, and compilation itself is more and more often searched or learned, with neural policies guiding exact solvers [17]. In that setting the question of *how* a compiler represents its decisions is not an implementation detail. A representation that can state, exactly and compactly, which choices are legal, which are better, and why an option was discarded, supports a kind of search and a kind of explanation that a sequence of greedy commitments or a trained policy does not.

The Luminal compiler challenge [1] is a small and precise instance of this problem. A program is a list of operations in dependence order; the machine has five engines with fixed issue capacities, latencies that delay when a result may be read, and a scratchpad of 256 words of 32 bits in which

eight-word vectors must be aligned. The compiler must decide when each operation issues and where each value lives, and it is scored by the geometric mean, over programs, of how many times fewer cycles and scratch words it needs than a serial compiler. The two decisions are coupled: packing a schedule tightly lengthens the time during which values overlap, and packing memory tightly constrains when values may be written.

We answer the challenge with a compiler in which every decision is the witness of an exact query over index sets. This paper is also an argument. The index-set method is *work in progress*, built on the author’s earlier published work on algorithmic information dynamics [2, 3], and the Luminal machine is a controlled setting in which to ask whether the method earns its keep against conventional engineering. We therefore report the whole programme and not only its last stage: the external benchmarks, our first and second versions, the intermediate stages and ablations, the final proposal, and the attempts that failed.

Contributions.

1. A compiler whose first answer is built entirely from exact index-set queries (Section 4) and whose improvement phase reopens related windows of decisions, confines each question to the region where a strict improvement can exist, deletes impossible options with replayable certificates and searches exactly (Section 5).
2. A shared branch state that lets each search node reuse the facts derived at its parent while reproducing its predecessor’s certificate stream in the same order; its decisions are identical at equal work on every tested input, and it is measurably faster (Section 8).
3. A worked example that runs every compared method on one public program and renders what each one actually did (Section 6).
4. A complete account of the measured gains, costs, losses and negative results, with each number tied to its comparator, population and interval (Sections 8 and 9, and Supplementary Section S10).

Four kinds of evidence. The final round produced evidence of four kinds, which we keep apart throughout. (a) A historical registered result: a pre-registered cost model predicted a conservative ratio of 0.8268 against a gate of 0.80 and stopped the round with NO_JUSTIFIED_MECHANISM; review later found a scope defect in one of its terms. (b) A post-hoc development forecast after that defect was corrected, of 0.7864 (conservative) and 0.6061 (point); it is neither a measurement nor an interval. (c) A kernel-only replay of captured propagation work, with a median speed-up of 2.3649 over 30 workloads (range 0.920 to 4.361); it is not a compiler speed-up. (d) Real compiler measurements: a development stage on 100 programs and a confirmation on 200 fresh programs. Only the confirmation is inferential.

2 The machine and the objective

The experiment pins the challenge repository at a fixed revision [1]. Inputs are straight-line operations in single-assignment dependence order, with static buffer ranges and supplied test cases. Each operation must be emitted exactly once; there is no rewriting, spilling or extra movement. The load, scalar and vector engines may each issue 2 operations per cycle and the store and flow engines 1. If operation i produces an argument of j , then $t_j \geq t_i + \ell_i$, where ℓ_i is the latency of i . Overlapping memory accesses to one buffer keep their source order when at least one of them is a store.

For a value v written by operation $p(v)$, its lifetime is the inclusive interval $I_v = [b_v, e_v]$ with $b_v = t_{p(v)} + \ell_{p(v)}$ and e_v the latest of b_v and the issue cycles of its readers. Two values may share

scratch words only if their lifetimes are disjoint. Scalars take one word; vectors take 8 consecutive words starting at a multiple of 8. The scored cycle count C is the number of emitted bundles, and the footprint S is the largest allocated end address. With (C_p^0, S_p^0) the serial compiler’s metrics for program p , the score over m programs is

$$Q = \left(\prod_{p=1}^m \frac{C_p^0 S_p^0}{C_p S_p} \right)^{1/(2m)}, \quad (1)$$

so that, for one program, maximising the score is the same as minimising the product $J = C \times S$. Every search in this paper minimises J . Only the 8 public programs are available; no private-grader result exists.

3 The methods compared

Several compilers appear in this paper, and they play different roles. Table 1 names each one, what it decides, how, and where it lives. The serial compiler is the yardstick that defines the score. The challenge’s starter compiler is the provided starting point; it places vectors and then scalars at fresh addresses and issues one operation per bundle, and on all eight public programs it reproduces the serial compiler’s cycles and footprint exactly, so its score is 1.000000. The classical compiler is the external benchmark: critical-path list scheduling [19, 20], followed by aligned first-fit allocation over live intervals [21]. The rest are stages of our own programme, from the first direct-index compiler to C1.

Method	Role	What it decides, and how	Source or frozen snapshot
Serial	yardstick; score one by definition	one operation per bundle, a fresh address per value; a fixed rule	<code>.reference/machine.py</code> , <code>serial_compile</code>
Starter	provided starting point	fresh addresses, vectors first; one operation per bundle; a fixed rule	<code>.reference/compiler.py</code>
Classical	external benchmark	critical-path list scheduling, then aligned first fit over inclusive live intervals in three fixed orders; greedy, never revisits a decision	<code>common.py</code> , <code>classical_compile</code> , <code>allocate</code>
Original direct (Phase 1)	our first version	each cycle and address is the smallest member of an exact cube query; a bounded four-operation joint-query optimiser	<code>direct_compiler.py</code> , <code>direct_optimizer.py</code> ; export d14bf39b450aaa5f
A0, structural encoding	our second version	decisions encoded as ranks among legal options; windowed search against target rectangles	<code>research/structural_search.py</code> ; run <code>recovery_campaign_20260923_r3</code>
A1–A4, <code>cap512_wider</code> , <code>cell_a4cat_dfs</code>	intermediate stages and ablations	each adds one mechanism: no query cap, product domains, certified propagation, a multiscale catalogue with discrepancy or depth-first traversal	<code>research/objective_index_search.py</code> , <code>research/next_round_search.py</code>
R0	accepted baseline before the last round	A4’s catalogue with depth-first search and repaired accounting	<code>research/efficiency_search.py</code> (d0fcd441886cc5c3); export 74c87b69e6306359
C1	final proposal	R0 with a shared branch state in the search	<code>research/third_round_candidate.py</code> ; export 58ee75059dcbdd9f

Table 1: Every compiler that appears in the paper. The first three are fixed external references; the rest are stages of one programme. Hashes are the first sixteen hexadecimal digits of the SHA-256 of the file; the full hash of C1 is in Supplementary Section S9.

Comparisons are made only between pairs that were actually measured together. C1 was measured against R0; R0’s predecessors were measured against A0, the accepted Phase 1 optimiser and the classical compiler in their own rounds. Where a pair was never measured directly, for example C1 against the classical compiler on a fresh population, we say so and do not multiply ratios across rounds.

4 Decisions as exact queries over index sets

4.1 Cubes and covers

Fix n Boolean coordinates with weights 2^i and let $U = 2^n - 1$. A *cube* is a pair of integers (a, f) with $a \& f = 0$: the anchor a fixes the coordinates equal to one, and the free mask f marks the coordinates that may take either value. It stands for the set

$$\llbracket(a, f)\rrbracket = \left\{ a + \sum_{i: f_i=1} \epsilon_i 2^i : \epsilon_i \in \{0, 1\} \right\}, \quad (2)$$

of size $2^{\text{popcount}(f)}$, whose smallest member is a itself. A *cover* is a finite union of cubes. An index encodes a whole assignment of decision fields, such as an issue cycle or a scratch address; it is not itself a physical address.

Proposition 1 (Intersection and minimum). *Two cubes (a, f) and (b, g) of width n intersect if and only if $(a \oplus b) \& (U \oplus (f \mid g)) = 0$, and then their intersection is the cube $(a \mid b, f \& g)$. The smallest member of a non-empty cover is the smallest anchor among its cubes.*

Proof. An index lies in both cubes exactly when it agrees with every fixed coordinate of each. A conflict is a coordinate fixed in both with different values, which is what the displayed mask detects. Otherwise the fixed coordinates of the intersection are the union of both sets of fixed coordinates, and a coordinate is free only when it is free in both. Free weights are positive and disjoint from the anchor, so a cube’s smallest member is its anchor, and the smallest member of a union is the least of these. $\square$

Set difference produces an exact disjoint cover with at most n cubes, and an integer interval decomposes into aligned binary blocks (Supplementary Section S1). A query is an AND/OR expression whose leaves are covers; a depth-first traversal returns a cube only when every predicate has been discharged, a timeout or count limit is reported as unknown and never as unsatisfiable, and an exhausted search proves unsatisfiability only within the declared domain. The representation is not free of limits: exact flat covers of parity need exponentially many cubes, a lower bound we prove and state alongside every compression claim (Supplementary Section S2).

4.2 The first answer

The first answer processes operations in order. For each operation it forms the cover of cycles between its precedence bound and a safe horizon, subtracts the cycles on which its engine is already full, and takes the smallest survivor. It then computes exact lifetimes and, for each value in a fixed order, forms the cover of aligned addresses, subtracts the blocks held by values whose lifetimes overlap, and again takes the smallest survivor. No serial or classical schedule seeds it, and no external solver is used. Figure 1 shows both queries as the real code evaluates them on the worked example of Section 6.

5 From the first answer to C1

The first answer never revisits a decision. Everything after it does, and it does so by asking a sequence of exact questions, each of which is small enough to answer completely. Figure 2 follows one epoch of this process on the worked example.

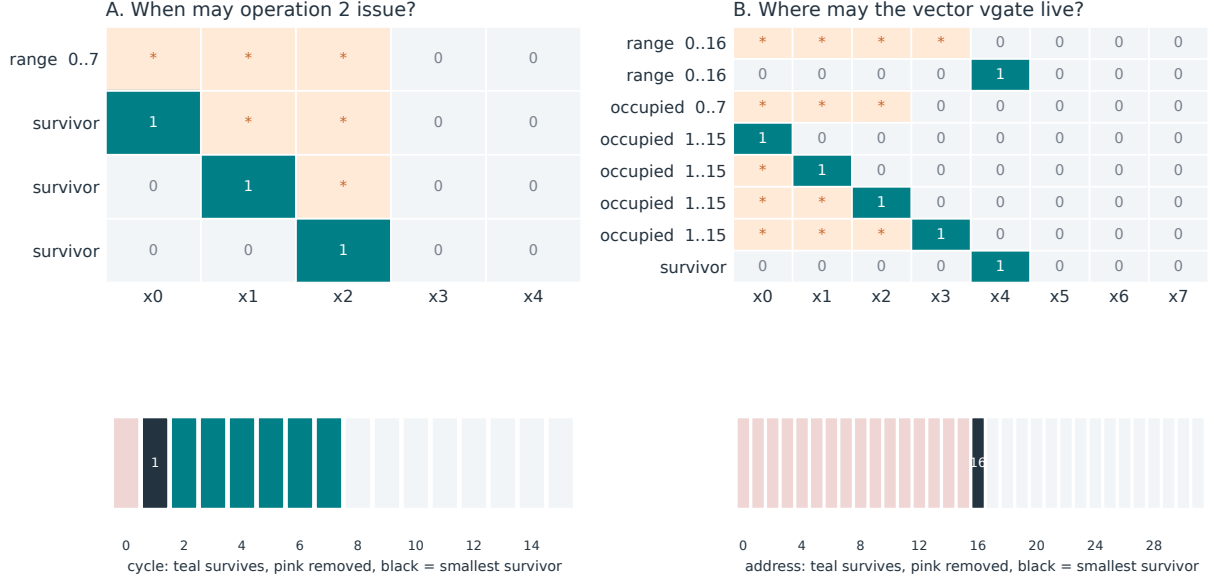


Figure 1: Two queries of the first answer, as the code evaluates them on `05_mixed_broadcast`. Each row of the upper panels is a cube over the coordinates $x_0, x_1, \dots$ (teal 1, grey 0, orange * free). (A) Operation 2 may issue in cycles 0 to 7, one cube; cycle 0 is removed because the load engine is already full there, which leaves 3 cubes whose smallest member, cycle 1, is chosen. (B) The vector `vgate` may start at any aligned address up to 16; the blocks held by the two vectors whose lifetimes overlap it are removed, and the smallest surviving aligned address, 16, is chosen. The lower strips expand the same sets for reading; the implementation never enumerates them. Every cover shown is recorded from the real run and checked for exactness before rendering.

Windows. Reopening every decision at once is hopeless, and reopening one at a time finds almost nothing, because useful changes move several operations together. Following Shaw’s large neighbourhood search [12], C1 builds a catalogue of *windows*: groups of related operations whose issue cycles may move within a radius, with everything else held fixed. The catalogue mixes the four-operation windows of the first version, the k latest-issued operations (which decide C), the k operations around the memory peak (which decide S), contiguous blocks of k , and the whole program when it is small.

Product caps. For a window, the fixed decisions alone give lower bounds L_C and L_S on any completion. A strict improvement needs $CS \leq J_0 - 1$, hence $C \leq \lfloor (J_0 - 1)/L_S \rfloor$ and $S \leq \lfloor (J_0 - 1)/L_C \rfloor$. Every strict improvement lies in this box, so the domain may be cut to it without losing any; when a cap falls below its lower bound the window is proved empty. This replaced the target rectangles of the first version, which leave parts of the region below the hyperbola unexamined.

Certified propagation. Inside the box, each reopened operation has an ordered list of candidate cycles whose first entry is its current cycle. Propagation removes values that cannot appear in any solution, and every removal emits a *certificate*: a tuple that anyone holding the program can check in one line. A precedence certificate records that v reads u with lag ℓ , so that v cannot issue before $\min D_u + \ell$ (PL) and u cannot issue after $\max D_v - \ell$ (PU); capacity certificates remove cycles whose slots are all occupied (EF) or prove a branch impossible (EO). This discipline of explained propagation is taken from lazy clause generation [15]; clause learning itself is not used.

Exact search and the controller. What propagation cannot rule out, a depth-first branch and bound decides, pruning a node when a lower bound on J is no better than the best known. A

controller keeps 8 questions resident and gives each a slice of at most 2 ms or 2,048 nodes in turn. The first question to find a strict improvement wins: its answer is validated by the challenge’s own checker, becomes the new incumbent, and a new epoch begins with a fresh catalogue. This is the combinatorial approach to scheduling and allocation of Castañeda Lozano and colleagues [14], applied to small neighbourhoods of one incumbent rather than to a whole function.

The shared branch state. In R0 each child node copies its parent’s domains and recomputes the whole fixed point, although it differs from the parent by one decision. C1 keeps at each node an immutable branch state (its domains and the facts derived from them) together with a change record of which domains lost their extreme values or became single cycles, and each rule re-examines only the constraints that the change can reach (Figure 3). This is incremental, event-driven propagation, a well-established idea in constraint programming [16], and we claim no novelty for it. The contribution is narrower: an exact emulation of R0’s sweep order, so that C1 emits the same certificates in the same order, together with an empirical parity test of that property.

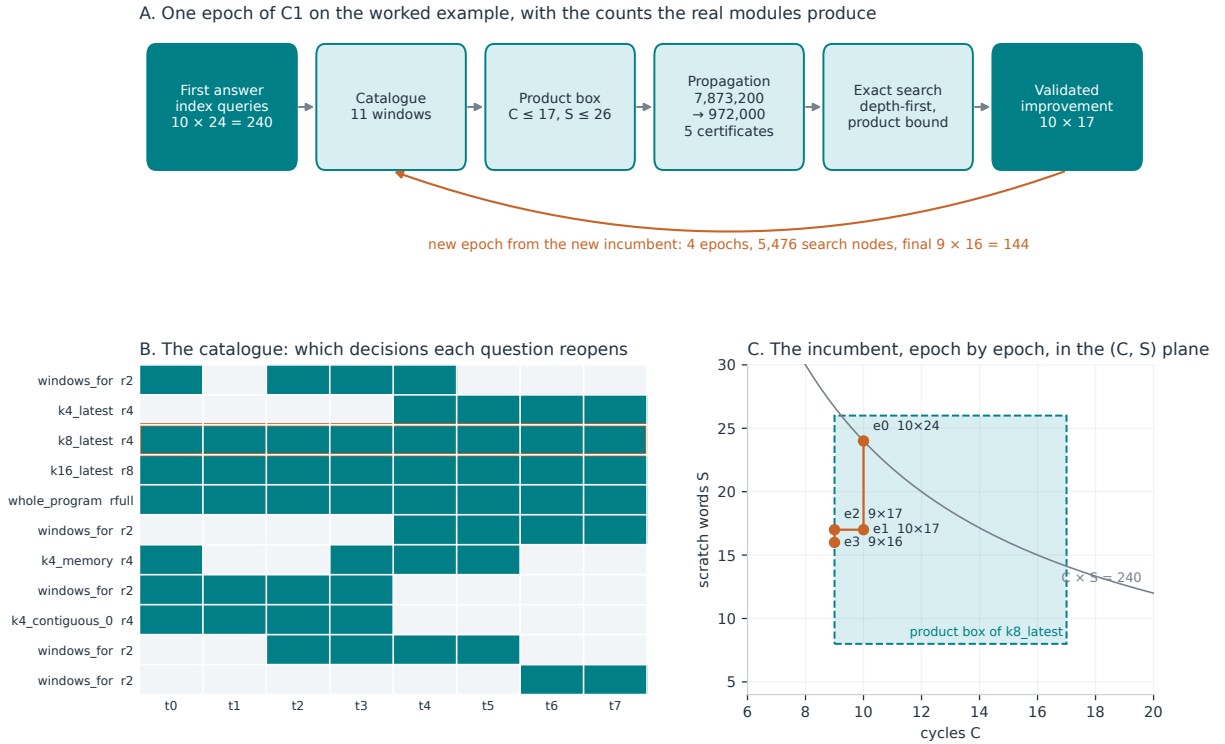


Figure 2: C1 on the worked example. (A) One epoch as an operation network, annotated with the counts the real modules produce: 11 windows; the product box of the `k8_latest` window, $C \leq 17$ and $S \leq 26$; propagation that reduces the declared time combinations from 7,873,200 to 972,000 with 5 certificates; and the validated improvement. (B) The catalogue: each row is one question and each filled cell an operation it may move. (C) The incumbent in the (C, S) plane over 4 epochs and 5,476 search nodes at fixed work; the first improvement lies inside the product box. All counts are checked before rendering; they describe one program and are not evidence of average gain.

6 A worked example, method by method

We now run every method on one public program, `05_mixed_broadcast` (Figure 4). It has 8 operations: three loads, two `splats` that copy a scalar into all lanes of a vector, a vector multiply, a vector add and a store. Its trade-offs are visible at a glance: the vector load is slow, two vectors must be alive at once, and the order in which they are created decides how much scratch they

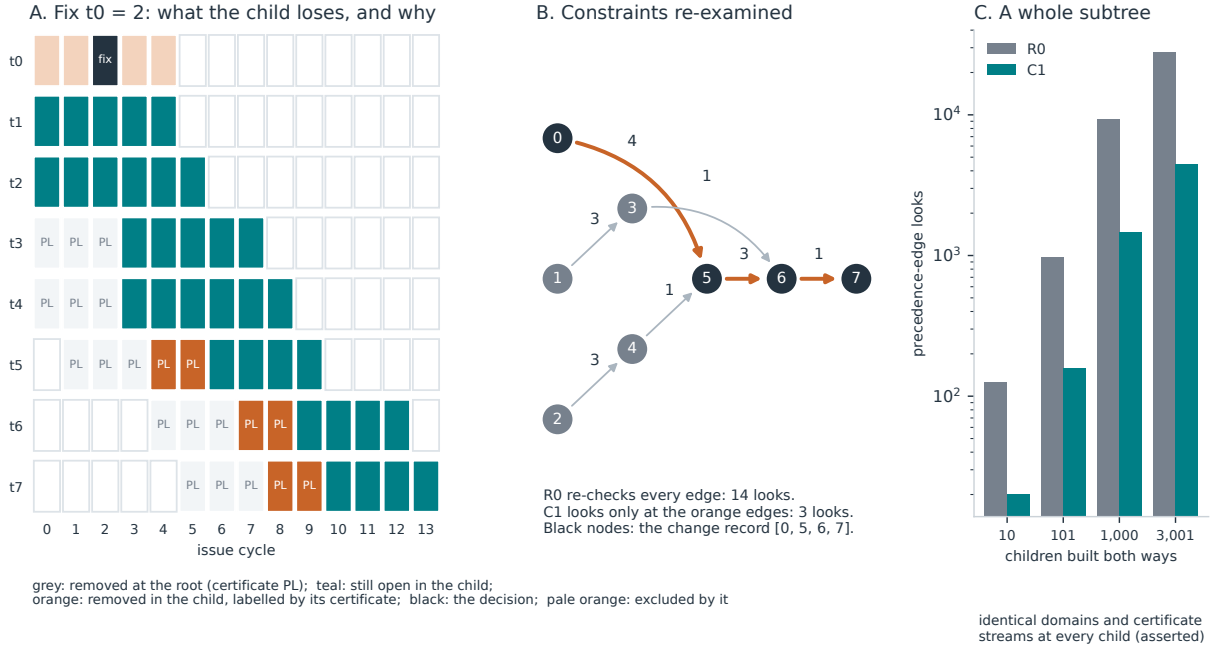


Figure 3: One propagation step, and the shared branch state. (A) Fixing operation 0 at cycle 2 in the `k8_latest` question: grey cells were removed at the root, orange cells are removed in the child and are labelled by the certificate that removed them. (B) The 7 precedence constraints; for this child R0 re-examines every one (14 edge looks) while C1 re-examines only the orange ones (3 looks). (C) Over a depth-first subtree of 3,001 children built both ways, R0 makes 27,874 edge looks and C1 4,440; the domains and the 1,655 certificates are identical at every child, which the generator asserts. The same counts are printed by notebook 04, and the generator checks that they agree.

need. Every number in this section describes this one program and is illustrative; none of it is evidence of an average gain.

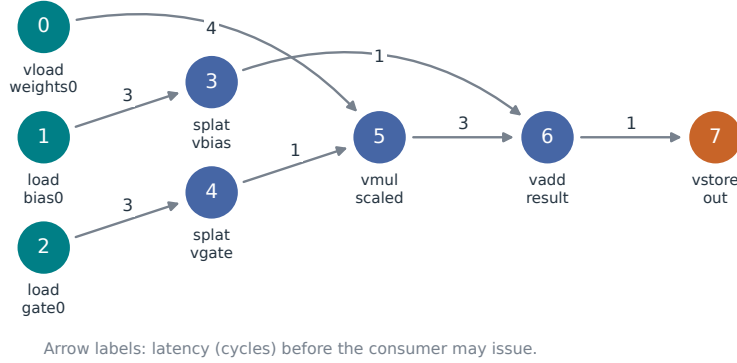
Figure 5 aligns the five methods' schedules and scratch layouts, and Table 2 gives their metrics. The serial and starter compilers follow a fixed rule and produce the same answer, 11 cycles and 42 words. The classical compiler follows local priorities: it issues the operations with the longest remaining path first and places each value at the lowest free address, reaching 9×17 . It never revisits a decision, and on this program its greedy choices happen to be good.

The original direct compiler states each choice as an exact query, so every cycle and address it emits is a witness of a stated set: the smallest member of a cover. Its answer, 10×24 , is *worse* than the classical one on both counts. The smallest-index rule places each vector as soon as it can, which keeps two eight-word vectors alive together, and the first version's four-operation windows cannot move enough operations at once to undo this. This is the program on which the first version lost, and it is why the later stages exist.

C1 starts from that same first answer and reopens it. Its first epoch, through the `k8_latest` window that releases all eight operations, keeps every cycle and cuts the footprint to 17 words. The second, through the same window, saves a cycle. The third, through an ordinary four-operation window that had nothing to offer from the first answer, saves one more word, ending at 9×16 (Figure 2C). The large move opened the way and a small one finished it. At fixed work R0 walks exactly the same path: the search traces and certificate streams of the two compilers have identical hashes on this program, as on every tested program. At a wall-clock allowance of 0.01 s the difference in speed becomes visible in the output: C1 reaches 10×16 and R0 10×17 . On the larger public program `08_vector_reduction`, with 19 operations, the same search lowers J from 480 to 384 within 9,999 nodes.

The contrast is between kinds of reasoning, not between scores on one program. The serial

A. 05_mixed_broadcast: operations and their dependences



B. Issue slots per cycle, by engine

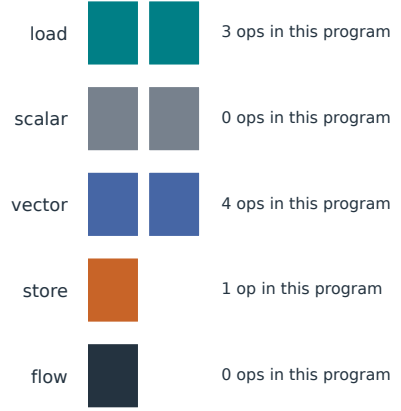


Figure 4: The worked example. (A) The dependence network of 05_mixed_broadcast; colour marks the engine and arrow labels the latency that must elapse before the consumer may issue. (B) The machine's issue slots per cycle. The generator checks that every arrow's label equals its producer's latency.

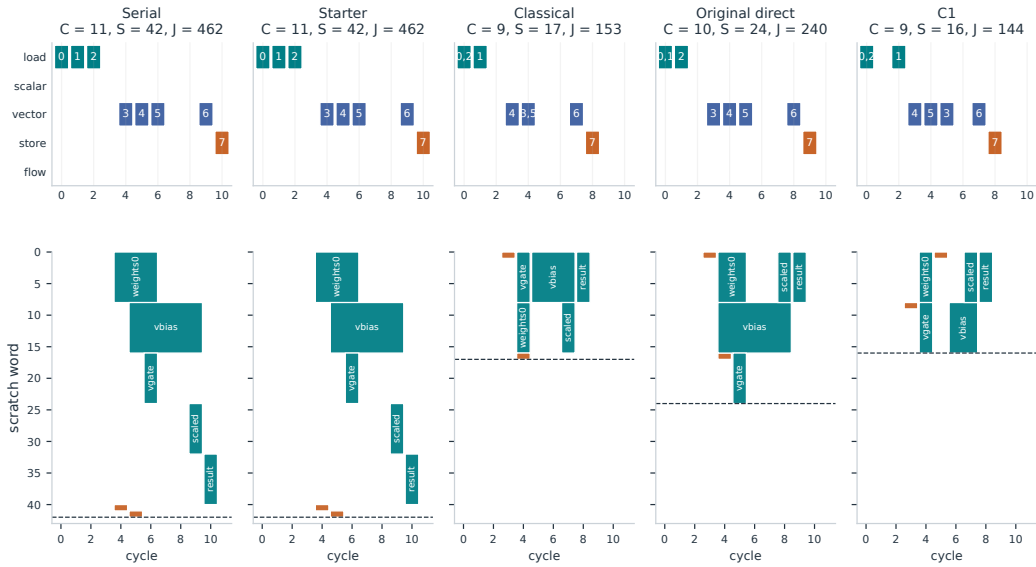


Figure 5: Five methods, one program. Top: the schedule as engine $\times$ cycle, with the identifiers of the operations issued in each slot. Bottom: the scratch layout, each value occupying its words from its write cycle to its last read; the dashed line is the footprint S . Every layout is produced by running the method, validated by the challenge's checker on every test case, and checked for capacity and for live overlaps before rendering.

Method	C	S	J	Compile call (frozen evidence)	Mechanism
Serial	11	42	462	0.015 ms, median of 3	fixed rule: one operation per bundle, a fresh address per value
Starter	11	42	462	not measured	fixed rule: the same as serial on this program
Classical	9	17	153	0.154 ms, median of 15	greedy priorities (critical path, first fit); never revisits a decision
Original direct	10	24	240	441.680 ms, median of 15	every cycle and address is the smallest member of a queried cover; the four-operation optimiser finds nothing
C1	9	16	144	233.5 ms, median of 5; completes its pass within the 1 s allowance	reopens related windows, confines each to its product box, propagates with certificates and searches exactly

Table 2: The worked example, one program. Compile times are medians from the frozen measurement runs, never measured here: serial from the Phase 1 comparison, classical and original direct from the Phase 1 timing run (full mode), C1 from the third-round public stage. They were recorded in different runs of one machine and are shown for scale only. The starter compiler was never timed.

and starter compilers apply a fixed rule. The classical compiler applies greedy local priorities and cannot undo a commitment. The direct method expresses each choice as an exact statement about a set, so any answer can be traced to the set it was drawn from. C1 then reopens related groups of decisions, confines each question to the only region where a strict improvement can exist, deletes impossible options with a local causal reason (“because v reads u with lag ℓ ”), searches the rest exactly, and reuses at each node the facts its parent derived.

7 Evaluation design

Populations. All generated programs come from the same generator in five families (scalar, vector, mixed, dependency and aliasing). The final round developed and selected its mechanism on development seeds and confirmed it on a fresh cohort of 200 programs, 40 per family, seeds 980000–980199, generated only after C1 had been frozen by hash. None of the cohort’s program, file or compilation-input hashes occurs in any earlier result file. Earlier rounds used their own fresh cohorts, which are never pooled.

Endpoints. Two endpoints were registered before measurement. The cost endpoint is the complete compile-call time at fixed work: the search is limited to 10,000 nodes on a logical clock that never advances, so both arms perform the same search. The quality endpoint is J at a real 0.1 s allowance. Each program was compiled 5 times by each arm in fresh processes. The estimator is the equal-family geometric mean of per-program median ratios, with a paired bootstrap over programs stratified by family (10,000 resamples, seed 2026092802, percentiles 1.25 and 98.75, a Bonferroni split across the two endpoints). The gates were an upper bound of at most 0.80 for cost and at most 1.01 for quality, with exact parity at fixed work.

Correctness. Every compilation in every stage was validated by the challenge’s evaluator on every supplied case. Both standalone exports pass a corpus of 142 programs and 277 cases. All 1,248 command-line runs of the exports were re-validated by the review with the challenge’s own evaluator.

Review. Codex, the designated lead reviewer, was unavailable for the final round, and the author authorised Claude Code to complete the review. The reviewer was therefore also the implementer, and the acceptance of C1 is a self-review. It recomputed every endpoint with new standard-library code that imports neither the reporter nor the auditor, re-validated the exports,

repeated the bootstrap under three further seeds (upper cost bounds 0.598, 0.597 and 0.598), and probed for order effects, drift, timer scope and heterogeneity. It also found two errors in the implementer’s own handoff, reported in Supplementary Section S8. An independent re-review would supersede it.

8 Results

8.1 The final proposal against its predecessor

C1 met both gates with a wide margin (Table 3). At fixed work the two compilers made the same decisions: 1,000 of 1,000 paired runs had identical search traces, incumbents, statuses and certificate streams (196 distinct traces over 200 programs; a median of 31,572 certificates per run). Of those pairs, 975 involve real search, because 5 programs need no search node at all. Parity holds on every tested input; it is not a proof for all programs. At that equal work C1’s complete compile call took 0.574 of R0’s time, with 97.5% interval [0.552, 0.598]; at the 0.1 s allowance its J ratio was 0.991, with interval [0.985, 0.996].

Endpoint (C1 / R0, 200 fresh programs)	Estimate	97.5% interval	Gate
Compile call at fixed work (10,000 nodes)	0.5743	[0.5518, 0.5978]	≤ 0.80
J at 0.1 s	0.9908	[0.9846, 0.9964]	≤ 1.01
Parity at fixed work	1,000 / 1,000 pairs identical		exact

Table 3: The registered endpoints of the final round, with the estimator and bootstrap described in Section 7. Both are ratios C1/R0; lower is better. The independent recomputation reproduced both estimates exactly.

The descriptive probes support the same reading. Adding the solver-module import time gives a cost ratio of 0.601, and whole-process time including interpreter start gives 0.649; the shared base compile and validation lie inside both arms and pull the ratio towards one. Splitting pairs by which arm ran first gives 0.572 (539 pairs, R0 first) and 0.577 (461 pairs, C1 first), and the ratio shows no trend across time quartiles (0.567, 0.538, 0.598, 0.597). Per program the ratio ranges from 0.264 to 1.076 (median 0.554), and C1 is slower on 7 of 200 programs (Figure 6). The saving grows with the size of the search, as a mechanism that reuses parent facts at every node predicts: 0.730 on the smallest third of programs by search size, 0.527 on the middle third and 0.491 on the largest. In wall-clock mode C1’s compile call is never longer (geometric ratios 0.984, 0.854 and 0.755 at allowances of 0.01, 0.1 and 1 s), so its quality is not bought with extra time. At the 0.1 s allowance C1 is better on 123, equal on 871 and worse on 6 of the paired runs; per program, 25 are better, 173 equal and 2 worse.

This is an engineering property of one Python implementation, measured on one local machine at equal search work. It is not a result about computational complexity, not a private-grader result, and not a comparison with the compile time of the classical compiler. Memory is a cost: the peak traced allocation of C1 relative to R0 had a median of 1.051 and a maximum of 1.302, measured on 30 development programs only; no memory measurement exists for the confirmation cohort.

8.2 Speed changes the path

Decision parity holds at equal work. Under a real clock it does not hold program by program, and the reason is instructive. The controller gives each resident question a slice of 2 ms or 2,048 nodes, and the first strict improvement ends the epoch. C1 expands more nodes per slice, so a different question can be the first to improve, and from there the two searches follow different paths. At a 1 s allowance C1 is worse than R0 on 17 of the paired runs, spread over 4 programs; it

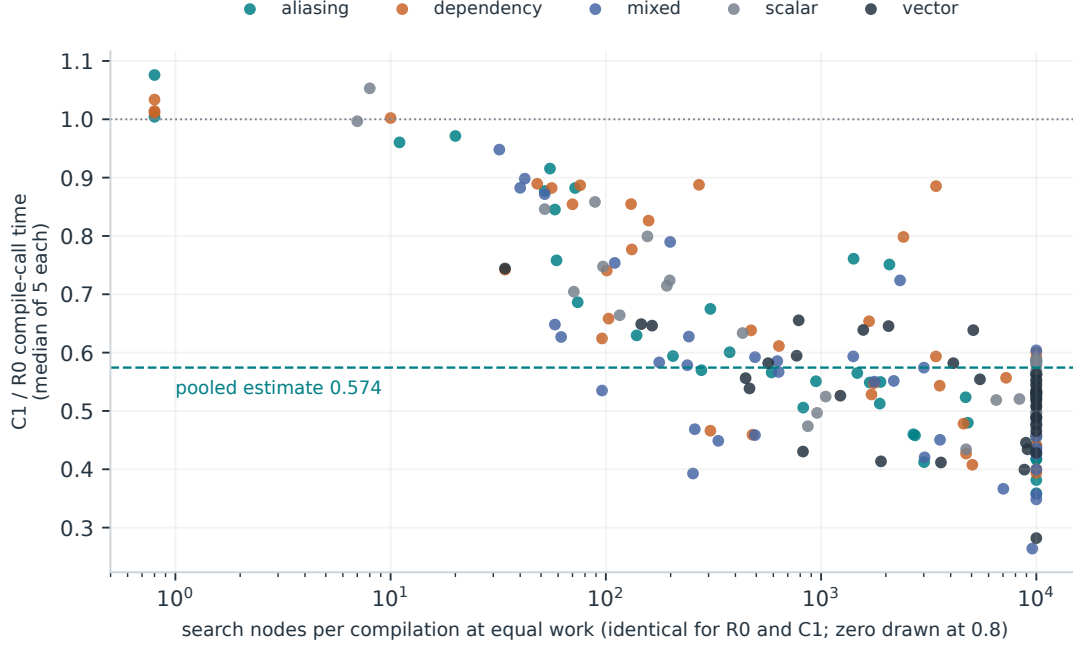


Figure 6: C1/R0 compile-call time at equal work, one point per fresh program, against the number of search nodes, which is identical for both arms. The dashed line is the confirmatory estimate. Ratios near one on tiny searches reflect fixed costs that both arms share; the saving grows with the search. The generator checks the minimum, maximum, median and number of programs above one against the independent recomputation.

is deterministic across repetitions on those programs, and 10 of the losses involve no unresolved question on either side. On program 980183 both start from the same first answer, 22×44 ; R0 ends at $22 \times 33 = 726$ and C1 at $22 \times 42 = 924$, in all 5 repetitions (Figure 7). The quality claim is therefore distributional, and it is stated as “decision parity at equal work”, never as a claim that the two compilers decide alike under a wall budget, nor that C1 is never worse.

This is not a defect of C1; it is a property of anytime first-improvement local search on a rugged landscape. A faster search reaches a different basin, and a different basin can be worse. Being faster does not imply being better.

8.3 The whole programme against the benchmarks

Table 4 gives the public score of every method, and Table 5 the per-program metrics of the external benchmarks, our first version and C1. The public programs were visible throughout development, so these are exact results on eight fixed programs, not estimates for a population. On them the programme moved from 2.008466 (first version) to 2.187957 (C1 at 0.1 s), against 1.901379 for the classical compiler.

On fresh populations the picture is more nuanced, and every measured pair appears in Figure 8 and Table 6 with its own comparator. The closest measured comparison of our search with the classical compiler is from round one: at 0.1 s the A4 search found 10.58% lower geometric J than classical (98.33% interval 8.38% to 12.84%) on 200 fresh programs, winning on 138, tying on 42 and *losing on 20*, at 76.20 times classical’s compile time. In round two the depth-first variant that became R0 was descriptively 14.4% better than classical on another fresh cohort (146 wins, 39 ties, 15 losses, unadjusted). C1 itself was never measured against classical on a fresh cohort; we do not chain C1/R0 onto these ratios.

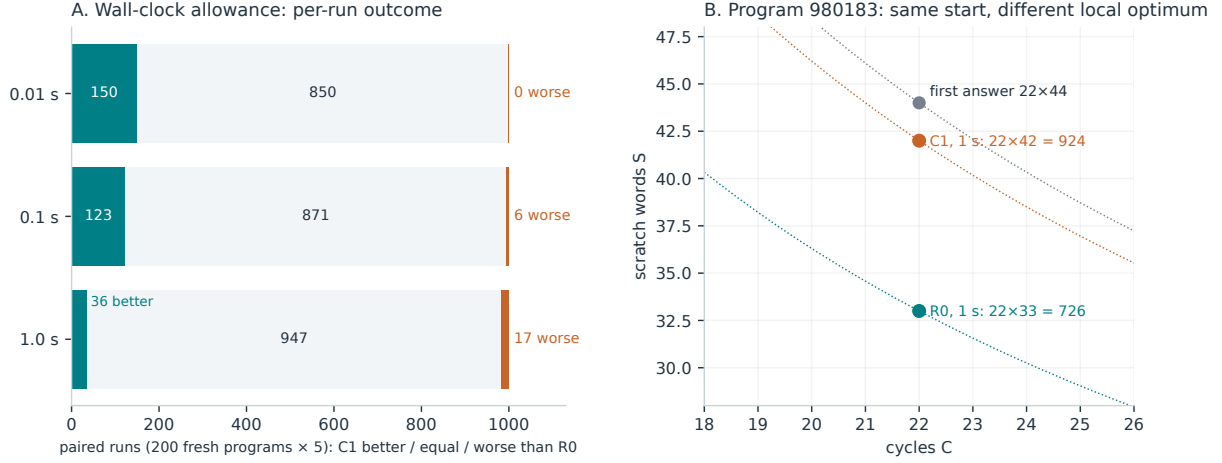


Figure 7: Quality is not monotone in speed. (A) Paired runs at three wall-clock allowances (200 fresh programs, 5 repetitions): C1 better, equal or worse than R0. (B) Program 980183 at 1 s: the same first answer, and two different local optima that each arm reached in every repetition; dotted curves are lines of constant J . The generator checks that the generated program is byte-identical to the one the frozen rows measured.

Method	Role	Allowance	Public score
Serial	yardstick	–	1.000000
Starter	provided start	–	1.000000
Classical	external benchmark	–	1.901379
Original direct (Phase 1)	our first version	default	2.008466
A0, structural encoding	our second version	0.1 s	2.032760
A4 (round one)	intermediate	0.1 s	2.088945
cap512_wider	earlier optimiser	0.1 s	2.102747
cell_a4cat_dfs (round two)	intermediate	0.1 s	2.187957
R0	accepted baseline	0.01 s	2.081045
R0		0.1 s and 1 s	2.187957
C1	final proposal	0.01 s	2.135275
C1		0.1 s and 1 s	2.187957

Table 4: Score (Equation 1) on the 8 public programs, relative to serial; higher is better. Every value was identical in every repetition of its run. Scores from different rounds are exact on the same programs, so they may be compared, but they are not population estimates; the round-two row for `cap512_wider` shows why (Section 9).

Program	$C \times S$				J			
	Serial	Classical	Direct	C1	Serial	Classical	Direct	C1
scalar_pipeline	18×15	14×6	14×6	14×6	270	84	84	84
scalar_dual_chain	12×9	9×5	8×3	8×3	108	45	24	24
vector_axpy	14×73	10×32	10×24	11×16	1022	320	240	176
vector_bitmix	16×89	10×40	10×40	13×24	1424	400	400	312
mixed_broadcast	11×42	9×17	10×24	9×16	462	153	240	144
parallel_memory	21×128	14×56	13×40	13×40	2688	784	520	520
scalar_selects	20×16	9×6	9×6	10×5	320	54	54	50
vector_reduction	19×137	12×40	12×40	12×32	2603	480	480	384

Table 5: Per-program metrics on the public programs. Direct is the original direct compiler (its bootstrap and full modes give identical public metrics); C1 is at the 0.1 s allowance, where R0 gives the same metrics. Bold marks the lowest J in each row. Values are constant across all repetitions.

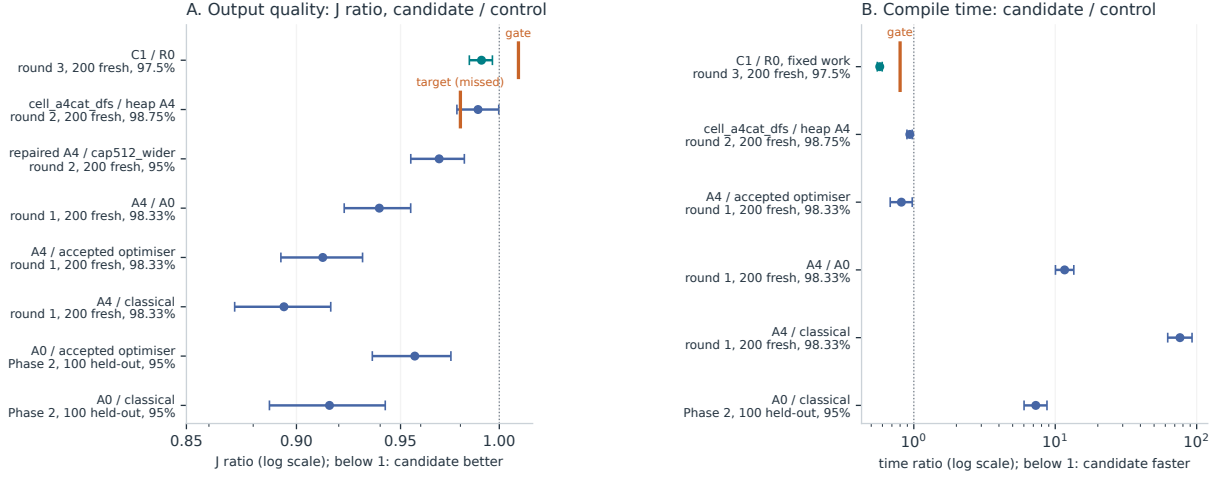


Figure 8: Every measured pair, each against its own control. (A) Output quality as the geometric J ratio of candidate to control; (B) compile time as the geometric ratio of candidate to control. Each row names its round, population and interval level. Orange bars are registered gates or targets: C1’s gates were met; the round-two practical target of 0.98 was missed (TARGET_NOT_REACHED). Rows are never combined across rounds.

Pair (candidate / control)	Population, allowance, interval	Quality	W / T / L	Time ratio
C1 / R0	round 3, 200 fresh, 0.1 s; 97.5%	J ratio 0.991 [0.985, 0.996]	123 / 871 / 6	0.574 ^a
cell_a4cat_dfs / heap A4	round 2, 200 fresh, 0.1 s; 98.75%	J ratio 0.989 [0.978, 0.9997]	29 / 161 / 10	0.937
cell_a4cat_dfs / cap512_wider	round 2, 200 fresh, 0.1 s; 98.75%	J ratio 0.959 [0.939, 0.9757]	70 / 116 / 14	0.916
repaired A4 / cap512_wider	round 2, 200 fresh, 0.1 s; 95%	J ratio 0.969; mean log +0.0312 [+0.0182, +0.0460]	55 / 123 / 22	0.978
A4 / A0	round 1, 200 fresh, 0.1 s; 98.33%	J 6.04% lower [4.50, 7.74]%	102 / 98 / 0	11.65
A4 / accepted optimiser	round 1, 200 fresh, 0.1 s; 98.33%	J 8.76% lower [6.86, 10.72]%	127 / 73 / 0	0.817
A4 / classical	round 1, 200 fresh, 0.1 s; 98.33%	J 10.58% lower [8.38, 12.84]%	138 / 42 / 20	76.20
A0 / accepted optimiser	Phase 2, 100 held-out, 0.1 s; 95%	mean log 0.0439 [0.0251, 0.0660]	26 / 74 / 0	0.073 ^b
A0 / classical	Phase 2, 100 held-out, 0.1 s; 95%	mean log 0.0882 [0.0592, 0.1194]	53 / 35 / 12	7.286
Original direct / classical	Phase 1, 100 generated, default	score geomeans 2.376597 vs 2.370890	–	657.5

Table 6: Every measured pair. Quality is reported in the statistic its protocol registered; “mean log” is the mean paired $\log(J_{\text{control}}/J_{\text{candidate}})$, positive when the candidate is better. W/T/L counts programs, except for C1/R0, where it counts paired runs. Time ratios are geometric means of per-program median compile times, candidate over control. ^aAt fixed work. ^bA0 stops early, so it finishes long before its allowance. No row may be multiplied by another.

8.4 What the gains cost

The method is slower to compile than the classical compiler, by a wide margin. In round one, A4 took 76.20 times classical’s compile time (98.33% interval 62.41 to 92.74) and 11.65 times A0’s. The first version, in its full mode, took 657.5 times classical’s time, and even A0, which stops early, took 7.286 times. C1 recovers part of the cost of its own search (Table 3), but nothing we measured suggests that it approaches classical compile time, and we make no such claim. The quality gains are also not uniform: A4 lost to classical on 20 of 200 fresh programs, the first version was worse than classical on `mixed_broadcast` (Table 5), and on generated programs the first version’s full mode was only marginally above classical (score geometric means 2.376597 against 2.370890). What the method offers is a trade: better output on average, a reason for every deletion, and a search that can be reopened, in exchange for compile time.

9 What did not work

The negative results are part of the evidence, and each keeps its own comparator.

- *The ladder.* Removing the query cap (A1) gained +0.0055 in mean log J over A0; product domains (A2) and certified propagation (A3) gained nothing in quality (−0.0011 and +0.0000), although each made compilation cheaper (0.82 and 0.83 times the previous step). Only the multiscale catalogue (A4) moved quality, by +0.0579, at 14.57 times A3’s compile time (Supplementary Section S3).
- *Catalogue, not traversal.* A two-by-two factorial on 100 development programs attributed A4’s gain to its catalogue (−0.0623 in log J , 95% interval [−0.0843, −0.0430]). The discrepancy-ordered traversal of Harvey and Ginsberg [13] made no measurable difference ([−0.0034, +0.0070]), and neither did the interaction.
- *A reversal between the public suite and the population.* On fresh programs repaired A4 beat the earlier `cap512_wider` optimiser (mean log J +0.0312, 95% interval [+0.0182, +0.0460], with 22 programs worse), while on the public suite the order reversed, 2.102747 against 2.088945. Neither result overrides the other; they answer different questions.
- *A missed target.* The round-two candidate `cell_a4cat_dfs` was better than A4 (J ratio 0.989) but missed both registered routes to a “better” claim (an upper J bound of 0.98, or a compile-time upper bound of 0.80 at no loss): `TARGET_NOT_REACHED`.
- *Two engineering attempts that stopped.* A worklist fixed point saved 4.8% of compile time against a gate of 10.0%, and a filter on the capacity rule was predicted to reach only 0.939 of the fixed-work time against a gate of 0.80: `NO_JUSTIFIED_OPTIMIZATION`.
- *The original registration of the shared branch state.* Its cost model gave 0.8268 and stopped the round; the review found that one term was measured over a narrower scope than the work it was meant to subtract. The correction is post hoc and is reported as a forecast (0.7864), not as a measurement.
- *Learning.* A depth-three schema tree [18], used in the spirit of learned guidance for exact search [17], lost to plain Hamming order on all 30 of 30 fixtures, and in 100 real compilations not one query collected the observations the learner needed (0 of 100). Round one’s learning hypothesis had failed by design: none of its 30 fixtures contained a test solution better than the training minimum. The per-query learner is retired for this architecture.
- *Structural encoding.* The feasibility campaign behind A0 did not meet its coverage gate and was recorded as inconclusive (Supplementary Section S7).

10 Why a different way of computing

The framing in this section is motivation and interpretation. The evidence is the set of measured gains, costs and intervals above, and nothing said here adds to it.

A lineage in work in progress. The index-set method grows out of the author’s research on algorithmic information dynamics and the programmability of integrated systems [2, 3], which in turn builds on Zenil and collaborators’ algorithmic information calculus and causal deconvolution [9, 10, 11]. Its broader form, index deconvolution, gives an exact finite representation of what a system can do, as sets written in index form, from which behaviour can be recovered, queried and explained. That method remains in development, and this paper neither describes nor depends on its construction. What the compiler uses is the narrower idea that a family of decisions can be stated exactly as a set of indices and queried without enumeration. It is a specialisation of these ideas to exact finite descriptions, and it is distinct from a computation of Kolmogorov complexity itself.

Algorithmic information. A cube is two integers that stand for a set of any size, and a cover is a short exact description of which choices are legal. The search asks, in effect, for the shortest exact statement of the region where a better answer can lie, which is the program-length view of information of Kolmogorov and Chaitin [4, 5, 6]. We compute no Kolmogorov complexity and no estimate of it; the link is conceptual and representational, not a measured quantity, and the parity bound of Supplementary Section S2 shows that the representation compresses only what is structured. The interest in simple exact rules that yield rich behaviour is consonant with Wolfram’s Principle of Computational Equivalence [7, 8], which we use as motivation and not as a theorem about compilers.

Causality. Every deletion of an option carries a certificate that states a local reason and can be replayed by anyone holding the program. Reopening a window and solving it again is an intervention on the schedule that holds the rest fixed, in the sense of the interventional calculus of the cited work [10]. The result is a compiler whose every decision can be audited: why a cycle was not used, which constraint forced a value to move, and which question produced an improvement. A trained policy that emits a schedule does not, by itself, offer this.

Complexity. The problem is combinatorial and its landscape is rugged. Large neighbourhoods of related decisions succeeded where local moves did not, which is the lesson of large neighbourhood search [12], and speed changed the trajectory of the search (Section 8.2), which is a hallmark of path dependence in complex search. Both findings are measured; their interpretation as features of a rugged landscape is ours.

11 Limitations

The final mechanism was developed and selected on development programs, its kernel draft preceded its specification, and its cost model was corrected after the outcome; only the fresh confirmation is inferential. All measurements come from one local machine and one Python implementation, and the speed-up is an engineering property of that implementation, not an asymptotic result. The public programs were visible throughout development, and there is no private-grader result. Memory was measured on development programs only. All generated programs come from one generator, so no claim transfers to real compiler workloads without further evidence. Parity at equal work is established on the tested inputs, not proved. The acceptance of C1 was a self-review. Finally, the challenge asks that the exercise and its solution

not be shared publicly; this manuscript is prepared to publication standard but is kept local, and any release requires that restriction to be resolved.

12 Conclusion

We set out to answer the Luminial challenge and to test whether a different way of computing is worth pursuing. The answer to the first is a compiler that is correct on every supplied case, that scores 2.187957 on the public programs against 1.901379 for a classical list scheduler, and whose final stage runs the same search in 0.574 of its predecessor’s time at equal work without losing quality at a fixed allowance. The answer to the second is more guarded and, we think, more interesting. Stating every decision as an exact query over index sets gave a compiler that can explain each of its choices, reopen any group of them, and prove when a question has no better answer; the gains it earned came from asking larger and better-shaped questions, and its costs are real and measured. As compilation moves towards accelerators that expose every decision and towards search and learning at scale, a representation that is exact, compact and auditable is, on this evidence, worth developing further.

Code, data and assistance

All figures and tables are regenerated from frozen evidence by the paper’s build; the worked example is produced by running the compilers themselves, and every number cited is recorded, with its source file and field, in the claim ledger of Supplementary Section S10. Claude Code assisted with implementation, the final review and the preparation of this manuscript; Codex reviewed the earlier rounds. The author is responsible for the claims and their interpretation.

References

- [1] Luminial. *Compiler challenge*, pinned source revision 573b8a85f4bdb8c3d8ba9f180d5f98dac875c902. <https://github.com/luminial-ai/interview/tree/573b8a85f4bdb8c3d8ba9f180d5f98dac875c902>.
- [2] A. Hernández-Espinosa, H. Zenil, N. A. Kiani and J. Tegnér, *Estimations of Integrated Information Based on Algorithmic Complexity and Dynamic Querying*, arXiv:1904.10393 [q-bio.NC] (2019). <https://arxiv.org/abs/1904.10393>.
- [3] A. Hernández-Espinosa, H. Zenil, N. A. Kiani and J. Tegnér, “Estimations of Integrated Information Based on Algorithmic Complexity and Dynamic Querying,” in *Handbook of Unconventional Computing*, Vol. 1: Theory, ed. A. Adamatzky (World Scientific, 2021), Chap. 5, pp. 171–220. https://doi.org/10.1142/9789811235726_0005.
- [4] A. N. Kolmogorov, *Three approaches to the definition of the concept of quantity of information*, *Problems of Information Transmission* 1(1), 3–11 (1965); English reprint, *Three Approaches to the Quantitative Definition of Information*, *International Journal of Computer Mathematics* 2(1–4), 157–168 (1968). <https://doi.org/10.1080/00207166808803030>.
- [5] G. J. Chaitin, *On the length of programs for computing finite binary sequences*, *Journal of the ACM* 13(4), 547–569 (1966). <https://doi.org/10.1145/321356.321363>.
- [6] M. Li and P. M. B. Vitányi, *An Introduction to Kolmogorov Complexity and Its Applications*, 4th ed., Texts in Computer Science (Springer, Cham, 2019). <https://doi.org/10.1007/978-3-030-11298-1>.

- [7] S. Wolfram, *A New Kind of Science*, Chap. 12 (Wolfram Media, Champaign, IL, 2002).
<https://www.wolframscience.com/nks/>.
- [8] S. Wolfram, “Notes for Chapter 12: The Principle of Computational Equivalence,” in *A New Kind of Science Online* (2002). <https://www.wolframscience.com/reference/notes/1128c/>.
- [9] H. Zenil, A. Schmidt and J. Tegnér, *Causality, Information, and Biological Computation: An Algorithmic Software Approach to Life, Disease, and the Immune System*, in *From Matter to Life: Information and Causality*, eds. S. I. Walker, P. C. W. Davies and G. F. R. Ellis (Cambridge University Press, 2017), pp. 244–280. <https://doi.org/10.1017/9781316584200.011>.
- [10] H. Zenil, N. A. Kiani, F. Marabita, Y. Deng, S. Elias, A. Schmidt, G. Ball and J. Tegnér, *An Algorithmic Information Calculus for Causal Discovery and Reprogramming Systems*, *iScience* 19, 1160–1172 (2019).
<https://doi.org/10.1016/j.isci.2019.07.043>.
- [11] H. Zenil, N. A. Kiani, A. A. Zea and J. Tegnér, *Causal deconvolution by algorithmic generative models*, *Nature Machine Intelligence* 1, 58–66 (2019). <https://doi.org/10.1038/s42256-018-0005-0>.
- [12] P. Shaw, *Using Constraint Programming and Local Search Methods to Solve Vehicle Routing Problems*, in *Principles and Practice of Constraint Programming – CP98*, eds. M. Maher and J.-F. Puget, LNCS 1520 (Springer, 1998), pp. 417–431. https://doi.org/10.1007/3-540-49481-2_30.
- [13] W. D. Harvey and M. L. Ginsberg, *Limited Discrepancy Search*, in *Proceedings of the 14th International Joint Conference on Artificial Intelligence (IJCAI-95)*, Vol. 1, pp. 607–613 (1995).
<https://www.ijcai.org/Proceedings/95-1/Papers/080.pdf>.
- [14] R. Castañeda Lozano, M. Carlsson, G. Hjort Blindell and C. Schulte, *Combinatorial Register Allocation and Instruction Scheduling*, *ACM Transactions on Programming Languages and Systems* 41(3), Article 17 (2019). <https://doi.org/10.1145/3332373>.
- [15] O. Ohrimenko, P. J. Stuckey and M. Codish, *Propagation via lazy clause generation*, *Constraints* 14(3), 357–391 (2009). <https://doi.org/10.1007/s10601-008-9064-x>.
- [16] C. Schulte and P. J. Stuckey, *Efficient Constraint Propagation Engines*, *ACM Transactions on Programming Languages and Systems* 31(1), Article 2 (2008).
<https://doi.org/10.1145/1452044.1452046>.
- [17] M. Gasse, D. Chételat, N. Ferroni, L. Charlin and A. Lodi, *Exact Combinatorial Optimization with Graph Convolutional Neural Networks*, in *Advances in Neural Information Processing Systems 32* (NeurIPS 2019). <https://arxiv.org/abs/1906.01629>.
- [18] L. Breiman, J. H. Friedman, R. A. Olshen and C. J. Stone, *Classification and Regression Trees* (Wadsworth, 1984).
- [19] R. L. Graham, *Bounds on multiprocessing timing anomalies*, *SIAM Journal on Applied Mathematics* 17(2), 416–429 (1969). <https://doi.org/10.1137/0117039>.
- [20] P. B. Gibbons and S. S. Muchnick, *Efficient instruction scheduling for a pipelined architecture*, in *Proceedings of the SIGPLAN ’86 Symposium on Compiler Construction*, pp. 11–16 (1986).
<https://doi.org/10.1145/12276.13312>.
- [21] M. Poletto and V. Sarkar, *Linear Scan Register Allocation*, *ACM Transactions on Programming Languages and Systems* 21(5), 895–913 (1999). <https://doi.org/10.1145/330249.330250>.
- [22] J. A. Fisher, *Very Long Instruction Word Architectures and the ELI-512*, in *Proceedings of the 10th Annual International Symposium on Computer Architecture*, pp. 140–150 (1983).
<https://doi.org/10.1145/800046.801649>.